Red Hat OpenShift AI Self-Managed 3.0

Experimenting with models in the gen AI playground

Experiment with models in the gen AI playground in Red Hat OpenShift AI Self-Managed

Last Updated: 2025-12-01

Red Hat OpenShift AI Self-Managed 3.0 Experimenting with models in the gen AI playground

Experiment with models in the gen AI playground in Red Hat OpenShift AI Self-Managed

Legal Notice

Copyright © 2025 Red Hat, Inc.

The text of and illustrations in this document are licensed by Red Hat under a Creative Commons Attribution–Share Alike 3.0 Unported license ("CC-BY-SA"). An explanation of CC-BY-SA is available at
http://creativecommons.org/licenses/by-sa/3.0/
. In accordance with CC-BY-SA, if you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, Red Hat Enterprise Linux, the Shadowman logo, the Red Hat logo, JBoss, OpenShift, Fedora, the Infinity logo, and RHCE are trademarks of Red Hat, Inc., registered in the United States and other countries.

Linux ® is the registered trademark of Linus Torvalds in the United States and other countries.

Java ® is a registered trademark of Oracle and/or its affiliates.

XFS ® is a trademark of Silicon Graphics International Corp. or its subsidiaries in the United States and/or other countries.

MySQL ® is a registered trademark of MySQL AB in the United States, the European Union and other countries.

Node.js ® is an official trademark of Joyent. Red Hat is not formally related to or endorsed by the official Joyent Node.js open source or commercial project.

The OpenStack ® Word Mark and OpenStack logo are either registered trademarks/service marks or trademarks/service marks of the OpenStack Foundation, in the United States and other countries and are used with the OpenStack Foundation's permission. We are not affiliated with, endorsed or sponsored by the OpenStack Foundation, or the OpenStack community.

All other trademarks are the property of their respective owners.

Abstract

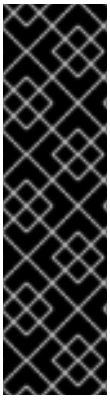
As a Red Hat OpenShift AI user, you can experiment with models in the gen AI playground in Red Hat OpenShift AI Self-Managed.

Table of Contents

PREFACE

Use the generative AI (gen AI) feature in OpenShift AI to evaluate, test, and interact with foundation and custom models in your project. You can test prompt engineering with Retrieval-Augmented Generation (RAG) and validate model behavior before using the model in an application.

CHAPTER 1. PLAYGROUND OVERVIEW



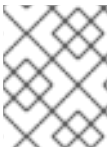
IMPORTANT

This feature is currently available in Red Hat OpenShift AI 3.0 as a Technology Preview feature. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see Technology Preview Features Support Scope .

The generative AI (gen AI) playground is an interactive environment within the Red Hat OpenShift AI dashboard where you can prototype and evaluate foundation models, custom models, and Model Control Protocol (MCP) servers before you use them in an application.

You can test different configurations, including retrieval augmented generation (RAG), to determine the right assets for your use-case. After you find an effective configuration, you can retrieve a Python template that serves as a starting point for building and iterating in a local development environment



NOTE

Important: The playground is a stateless environment. If you refresh your browser or end your session, all chat history and parameter settings will be lost.

1.1. CORE CAPABILITIES

The playground includes a core set of features:

- **Interact with models:** You can chat with both foundational and custom-deployed models.
- **Test with RAG:** You can test prompt engineering with document-based Retrieval-Augmented Generation (RAG) by uploading your own documents for the model to use as context.
- **Integrate MCP servers:** You can authorize and interact with approved Model Control Protocol (MCP) servers and their tools.
- **Export configurations:** You can export prompts and parameter configurations as a code template to iterate with in your local IDE.

CHAPTER 2. PLAYGROUND PREREQUISITES

Before you can configure and use the gen AI playground feature, you must meet prerequisites at both the cluster and user levels.

2.1. CLUSTER ADMINISTRATOR PREREQUISITES

Before a user can configure a playground instance, a cluster administrator must complete the following setup tasks:

- Ensure that OpenShift AI is installed on an OpenShift cluster running version 4.19 or later.
- Set the value of the **spec.dashboardConfig.genAiStudio** dashboard configuration option to **true**. For more information, see Dashboard configuration options.
- If using OpenShift AI groups, add users to the **rhods-users** and **rhods-admins** OpenShift group.
- Ensure that the Llama Stack Operator is enabled on the OpenShift cluster by setting its **managementState** field to **Managed** in the **DataScienceCluster** custom resource (CR) of the OpenShift AI Operator. For more information, see Activating the Llama Stack Operator.
- Configure Model Control Protocol (MCP) servers to test models with external tools. For more information, see Configuring model control protocol servers.

2.2. USER PREREQUISITES

After the cluster administrator completes the setup, you must complete the following tasks before you can configure your playground instance:

- You are logged in to OpenShift AI.
- If you are using OpenShift AI groups, you are a member of the appropriate user or admin group.
- Create a project. The playground instance is tied to a project context. For more information, see Creating a project.
- Add a connection to your project. For more information about creating connections, see Adding a connection to your project.
- Deploy a model in your project and make it available as an AI asset endpoint. For more information, see Deploying models on the model serving platform.

After you complete these tasks, the project is ready for you to configure your playground instance.

2.3. MODEL AND RUNTIME REQUIREMENTS FOR THE PLAYGROUND

To successfully use the retrieval-augmented generation (RAG) and Model Control Protocol (MCP) features in the playground, the model you deploy must meet specific requirements. Not all models offer the same capabilities.

2.3.1. Key model selection factors

Tool calling capabilities

The model must support tool calling to interact with the playground's RAG and MCP features. You must check the model card (for example, on Hugging Face) to verify this capability. For more information, see Tool calling in the VLLM documentation.

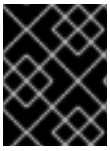
Context length

Models with larger context windows are recommended for RAG applications. A larger context window allows the model to process more retrieved documents and maintain longer conversation histories.

vLLM version and configuration

Tool calling functionality depends heavily on the version of vLLM used in your model serving runtime.

- **Version**: Use the latest vLLM version included in Red Hat OpenShift AI for optimal compatibility.
- **Runtime arguments**: You must configure specific runtime arguments in the model serving runtime to enable tool calling. Common arguments include (not exhaustive):
  - **--enable-auto-tool-choice**
  - **--tool-call-parser**
    For more information, see Tool calling in the VLLM documentation.



IMPORTANT

If these requirements are not met, the model might fail to search documents or execute tools without returning a clear error message.

2.3.2. Example model configuration

The following table describes an example configuration for the **Qwen/Qwen3-14B-AWQ** model for use in the playground. You can use this as a reference when configuring your own model runtime arguments.

Table 2.1. Example configuration for Qwen/Qwen3-14B-AWQ

Field	Configuration Details
Model	Qwen/Qwen3-14B-AWQ
vLLM Runtime	vLLM NVIDIA GPU ServingRuntime for KServe
Hardware Profile	NVIDIA A10G (24GB VRAM)
Custom Runtime Arguments	**--dtype=auto** **--max-model-len=32768** **--enable-auto-tool-choice** **--tool-call-parser=hermes** **--reasoning-parser=qwen3** **--gpu-memory-utilization=0.90**

2.4. CONFIGURING MODEL CONTROL PROTOCOL (MCP) SERVERS

A cluster administrator must configure and enable the Model Control Protocol (MCP) servers at the

platform level before users can interact with external tools in the Generative AI Playground. This configuration is done by creating a **ConfigMap** in the **redhat-ods-applications** namespace, which holds the necessary information for each MCP server.

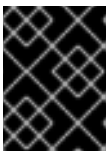
Prerequisites

- You have cluster admin privileges for your OpenShift cluster.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
  - Installing the OpenShift CLI for OpenShift Container Platform

Procedure

1. Create a file named **gen-ai-aa-mcp-servers.yaml** with the following YAML content. You can add multiple server entries under the **data:** field.

```
kind: ConfigMap
apiVersion: v1
metadata:
  name: gen-ai-aa-mcp-servers
  namespace: redhat-ods-applications
data:
  GitHub-MCP-Server: |
    {
      "url": "https://api.githubcopilot.com/mcp/x/repos/readonly",
      "description": "The GitHub MCP server enables exploration and interaction with repositories, code, and developer resources on GitHub. It provides programmatic access to repositories, issues, pull requests, and related project data, allowing automation and integration within development workflows. With this service, developers can query repositories, discover project metadata, and streamline code-related tasks through MCP-compatible tools."
    }
```



IMPORTANT

The **ConfigMap** key (**GitHub-MCP-Server**) is case-sensitive and must be unique. The content provided under this key must be valid JSON format.

2. Apply the **ConfigMap** to the cluster by running the following command:

```
oc apply -f gen-ai-aa-mcp-servers.yaml
```

Verification

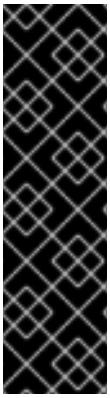
- Confirm that the **ConfigMap** was successfully applied by running the following command:

```
oc get configmap gen-ai-aa-mcp-servers -n redhat-ods-applications -o yaml | grep GitHub-MCP-Server
```

- The output should contain the key name, confirming its successful creation:

```
GitHub-MCP-Server: |
```

2.5. ABOUT THE AI ASSETS ENDPOINT PAGE



IMPORTANT

This feature is currently available in Red Hat OpenShift AI 3.0 as a Technology Preview feature. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

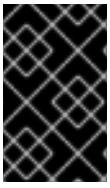
For more information about the support scope of Red Hat Technology Preview features, see Technology Preview Features Support Scope .

The **AI asset endpoints page** is a central dashboard for managing the generative AI assets available for you to use within your project.

The page organizes assets into two categories:

- Models: Lists all generative AI models deployed in your project that have been designated as available assets. For a model to be available, you must select the **Add as AI asset endpoint** check box when deploying it. For more information, see Deploying models on the model serving platform.
- MCP Server: Lists all available MCP servers associated with your project.

The primary purpose of this page is to provide a starting point for using these assets. From here, you can perform actions such as adding a model to a playground instance for testing.



IMPORTANT

The assets listed on the **AI assets endpoint** page page are scoped to your currently selected project. You will only see models and servers that are deployed and available within that specific project.

CHAPTER 3. CONFIGURING A PLAYGROUND FOR YOUR PROJECT

Configure a generative AI (gen AI) playground for your project, so that you can interact with your deployed generative AI models and connect to backend servers, such as the Model Control Protocol (MCP).

Prerequisites

- You have created a project.
- You have deployed a model in your project and added your model as an AI asset endpoint.
- If your cluster administrator has configured Model Control Protocol (MCP) servers, they are accessible within your OpenShift environment.

Procedure

Follow these steps to configure the playground:

1. Perform one of the following actions:
   a. From the OpenShift AI dashboard side navigation menu, click **Gen AI studio → Playground**.
      i. Select the project containing your model deployment from the **Project** drop-down list.
      ii. Click **Create playground**.
      The **Configure playground** dialog opens.
   b. From the OpenShift AI dashboard side navigation menu, click **Gen AI studio → AI asset endpoints**.
      i. Select the project containing your model deployment from the **Project** drop-down list.
      ii. Click the **Models** tab.
      iii. Locate the model that you want to create a playground for, and then click **Add to playground**
      The **Configure playground** dialog opens.
2. Select the check box next to the model deployment you want to interact with in this playground instance.
3. Click **Create**.
   Wait for the playground interface to finish loading.
4. Optional: Expand the **MCP servers** section. Select the checkbox for the appropriate MCP server instance to connect to.

Verification

- The playground interface loads successfully.
- The **Model details** section displays information about the selected model deployment.
- The selected MCP server shows a successful connection status within the **MCP servers** section.

CHAPTER 4. TESTING BASELINE MODEL RESPONSES

Use the playground to test and evaluate your model's baseline responses.

Prerequisites

- You have created a playground for your deployed model.

Procedure

To test your model, follow these steps:

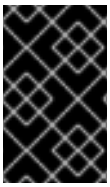
1. From the OpenShift AI dashboard, click **Gen AI studio → Playground**.
2. From the model list, select the model that you want to test.
3. Adjust the following model parameters as needed:
    a. **Temperature**: Control the randomness of the model's output. Use values between 0 and 2. The temperature value directly influences creativity: * **Values near 0 (0-0.3)** Produce deterministic and factual responses, and are recommended for objective or factual tasks. * **Values around 0.7**: A common default for balanced output. * **Values near 1 (0.7-1)** Increase creativity and randomness, and are recommended for generative or creative tasks. * **Values above 1 (such as 2)** Typically produce incoherent output.
    b. **Streaming**: Show the LLM's response as it is being generated. This is helpful for testing model latency and seeing the model's progress in real time. When streaming is off, the full response will not render until it is complete.
    c. **System instructions**: Review or edit the text to define the context, persona, or instructions for the model. The playground provides a default prompt.
4. In the chat input field, type a query.
5. Click **Send**.
6. Observe the model's response.

Verification

- The model provides a response based on its general knowledge or pre-trained data.

CHAPTER 5. TESTING YOUR MODEL WITH RETRIEVAL AUGMENTED GENERATION (RAG)

You can enhance your model's responses by providing it with contextual information from your own documents using retrieval augmented generation (RAG). You can upload documents to the vector database associated with the playground to provide context for your model's responses.



IMPORTANT

The RAG feature of the gen AI playground is currently configured to work only with an inline vector database. There is currently no mechanism to configure the playground to connect this RAG feature to an external or remote vector database.

Prerequisites

- You have configured a playground for your project.
- You have the document files ready to upload. The supported file formats are PDF, DOC, or CSV. You can upload up to 10 files, with a maximum size of 10MB per file.

Procedure

1. From the OpenShift AI dashboard, click **Gen AI studio → Playground**.
2. In the **Playground** interface, click the toggle in the **RAG** section and then expand the section.
3. Click **Upload**.
   The **Upload files** dialog opens.
4. Drag and drop your file or click to browse and select a file from your local system.
5. Optional: Adjust the **Maximum chunk length** and **Chunk overlap** and **Delimiter** values as needed for your document type. For more information about these settings, see Understanding RAG settings.
6. Click **Upload**.
   Wait for the file to finish processing. A **Source uploaded** notification appears, and the file is listed under **Uploaded files**.
7. Repeat these steps to upload additional files if needed.
8. In the **System instructions** field, review or edit the text to define the context, persona, or instructions for the model. The playground provides a default prompt.
9. In the chat input field, ask a question related to your documents that the model would not know otherwise.
10. Observe the model's response.

TIP

If a model is reluctant to use the RAG feature (its knowledge search tool), you can modify the prompt in the **System instructions** field to explicitly guide its behavior.

You can refine the system prompt by including directives such as:

- **To force use:** "You **MUST** use the **knowledge_search** tool to obtain updated information."
- **To specify context:** "Always search the knowledge base before answering questions about company policies, recent events, or specific documentation."

This ensures the model actively utilizes the available RAG documents rather than relying solely on its pre-trained data.

Verification

- The model retrieves information from the uploaded documents to answer the question.

5.1. UNDERSTANDING RAG SETTINGS

When you upload a document for retrieval augmented generation (RAG), you can configure the following settings to optimize how the text is processed.

Maximum chunk length	The maximum word count for each text section ("chunk") created from your uploaded files. • **Smaller chunks** are recommended for precise data retrieval. • **Larger chunks** are recommended for tasks requiring broader context, such as summarization.
Chunk overlap	The number of words from the end of one text section (chunk) that are repeated at the start of the next one. This overlap helps maintain continuous context across chunks, improving model responses. For example, the following sentence is chunked differently depending on the chunk overlap: "Chunk overlap can improve the quality of model responses." **Maximum chunk length = 4, Chunk overlap = 1** > Chunk overlap can improve > improve the quality of > of model responses. **Maximum chunk length = 4, Chunk overlap = 0** > Chunk overlap can improve > the quality of model > responses.

Delimiter	A character or string that specifies where a text chunk should end. This helps define text boundaries alongside maximum chunk length and overlap, ensuring sentences or paragraphs remain intact. Examples of delimiters: ● **.** (period) – splits at sentence boundaries ● **\n** (newline) – splits at paragraph boundaries ● **;** (semicolon) – splits at clause boundaries For example, the following sentence is split as follows depending on the delimiter: "This is the first sentence. This is the second sentence." **Maximum chunk length = 4 , Chunk overlap = 0** > This is the first sentence. This is the second sentence. **Maximum chunk length = 4 , Chunk overlap = 0, Delimiter = 0** > This is the first sentence. > This is the second sentence.

CHAPTER 6. TESTING WITH MODEL CONTROL PROTOCOL (MCP) SERVERS

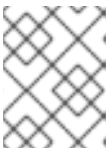
Authorize and interact with connected MCP servers to use their integrated tools directly from the playground chat.

Prerequisites

- You have deployed a model with tool-calling capabilities enabled in your project.
- You have configured a playground instance for your project.
- A cluster administrator has configured an MCP server, and the server is listed and available in the **MCP servers** section of your playground.

Procedure

1. In the **MCP servers** section, select the checkbox for the server that you want to use.
2. Click the **Auth** icon next to the server name.
   The **Authorize MCP server** dialog opens.
3. If the server requires a token, enter the token in the **Access token** field and click **Authorize**.
   A **Connection successful** message appears.



NOTE

Authorization tokens for MCP servers are stored only for the current browser session. If you close your browser, you must re-authorize the server.

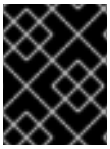
4. Click **Close**.
5. Click the **View tools** (wrench) icon for the same MCP server.
   A modal appears, listing all available tools for that server. You can copy a tool name to use in the chat.
6. In the chat input field, type a query that uses one of the available tools.
7. Click the **Send** button or press Enter.

Verification

- The AI bot responds, indicating it is using the tool.
- The bot provides the output from the tool.

CHAPTER 7. EXPORTING YOUR PLAYGROUND CONFIGURATION

Export your gen AI playground configuration as a Python code template so that you can use it in your local development environment, such as a notebook or IDE.



IMPORTANT

This code is a template and is not a runnable script. It provides a starting point that shows your configuration, including the model, MCP tools, and RAG files used.

Prerequisites

- You have configured your playground instance with the settings that you want to capture in your code template. This includes:
  - Selecting a model.
  - Setting model parameters, such as model temperature, to your desired values.
  - (Optional) Uploading files and enabling the **RAG** function.
  - (Optional) Authorizing and enabling any **MCP servers** that you intend to use.

Procedure

1. In the playground, configure your desired settings.
2. Click the **View code** button.
   A dialog opens, displaying a Python code template.
3. Click **Copy code**.
4. Paste the code into your local development environment.

Verification

- Review the pasted code in your local environment.
- Confirm that the template includes the correct model, MCP tools, and RAG files from your playground configuration.

CHAPTER 8. UPDATING YOUR PLAYGROUND CONFIGURATION

You can update the configuration of your playground instance to add new models, re-register models that were stopped, or change the existing configuration.

WARNING

Updating the playground configuration will permanently delete the inline vector database for **all users** in your project.

Prerequisites

- You have configured a playground for your project.

Procedure

1. From the OpenShift AI dashboard, click **Gen AI studio → Playground**.
2. Select the project containing your model deployment from the **Project** dropdown list.
3. In the upper-right corner of your playground, click the action menu (⋮) and select **Update configuration**.
4. On the configuration screen, select or clear the checkboxes for the models you want to make available.
5. Click **Update**.

Verification

- The playground configuration is updated with a new selection of models.

CHAPTER 9. DELETING A PLAYGROUND FROM YOUR PROJECT

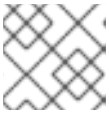
You can delete a playground instance from a project. This removes instance for all users who have access to that project.

Prerequisites

- You have configured a playground for your project.

Procedure

1. From the OpenShift AI dashboard, click **Gen AI studio → Playground**.
2. Select the project containing your model deployment from the **Project** drop-down list.
3. In the upper-right corner of your playground, click the action menu (⋮) and select **Delete playground**.



NOTE

This action deletes the playground for **every user** in the project.

4. Confirm the deletion.

Verification

- Confirm that the playground is deleted from the project.
- In the **Gen AI studio → AI asset endpoints** page, models no longer show the **Try in playground** button and instead show the **Add to playground** button.

CHAPTER 10. NEXT STEPS

You have successfully deployed and tested a model using the playground with RAG and MCP tools. For more information on the next steps, see the following resources:

Developing in an IDE

Working in your data science IDE
Learn how to access your workbench IDE (JupyterLab, code-server, or RStudio Server) to develop models.

CHAPTER 11. TROUBLESHOOTING PLAYGROUND ISSUES

If you encounter issues while using the playground, refer to the following scenarios and solutions.

11.1. THE CHATBOT THINKS INDEFINITELY

Problem After sending a query, the chatbot shows a thinking indicator but never returns a response.

Cause This issue often occurs when the query or the accumulated context exceeds the maximum context length (sequence length) configured for the model.

Solution

1. In the OpenShift AI dashboard, click the **Applications** menu and select **OpenShift Console**.
2. Navigate to your project's namespace.
3. Check the logs for the following pods:
   - The playground pod: **lsd-genai-playground-<id>**
   - The model serving pod: **<model-name>-predictor-<id>**
4. Look for errors related to context length limits or memory (OOM) constraints.

11.2. THE MODEL DOES NOT USE RAG DATA

Problem The model answers questions using its training data instead of searching the uploaded RAG documents.

Solution

Update the **System instructions** in the playground to explicitly force the use of the search tool.

- **Example:** "You MUST use the **knowledge_search** tool to obtain updated information."
- **Example:** "Always search the knowledge base before answering questions about company policies."

11.3. MCP SERVERS ARE MISSING FROM THE UI

Problem The **MCP servers** section is empty or not visible in the playground configuration.

Cause MCP servers must be configured at the cluster level by an administrator.

Solution

Contact your OpenShift AI administrator to configure the required MCP servers. Administrators can find a list of available servers in the Red Hat OpenShift AI documentation.

11.4. THE MODEL FAILS TO CALL MCP TOOLS

Problem The model attempts to use a tool but fails, or outputs raw XML tags (e.g., **<tool_call>**).

Cause

- The model does not support tool calling.
- The vLLM runtime arguments are missing or incorrect.
- **Known Issue:** Some models (e.g., **Qwen3-4B-Instruct**) may output raw tags if the correct reasoning parser is not available in the current vLLM version.

Solution

1. Verify the model supports tool calling on its Hugging Face model card.
2. In the model's deployment settings, ensure the following **Custom Runtime Arguments** are present:
   - **--enable-auto-tool-choice**
   - **--tool-call-parser**
3. If the model outputs **<think>** tags, you can hide them by adding /**no_think** to your prompt.